Ders 12 Genişletilmiş Notu: Öz Düzenleyici Haritalar, Pekiştirmeli Öğrenme ve Q-Öğrenme

Ders 12 Genişletilmiş Notu: Öz Düzenleyici Haritalar, Pekiştirmeli Öğrenme ve Q-Öğrenme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Eğitmensiz Öğrenmeye Kısa Dönüş
- 2. Rekabetçi Öğrenme ve “Kazanan Her Şeyi Alır”
- 3. Öz Düzenleyici Haritanın (SOM) Yapısı
- 4. SOM’da Öğrenme: Ağırlıkların Girişe Benzetilmesi
- 5. SOM Öğrenme Adımları
- 6. Sert ve Yumuşak Öğrenme, Komşuluk Fonksiyonu
- 7. Öğrenme Oranı ve Komşuluğun Zamanla Daraltılması
- 8. Örnek: SOM ile İki Kümeye Ayırma
- 9. Öğrenmeden Sonra Gelen Örnek ve Kararlılık-Esneklik İkilemi
- 10. Gauss Karışım Modeline Kısa Bakış
- 11. SOM Adımlarının Özeti
- 12. Pekiştirmeli Öğrenme
- 13. Pekiştirmeli Öğrenmenin Uygulamaları
- 14. Durum, Eylem ve Ödül
- 15. Markov Karar Süreci
- 16. Q-Öğrenme
- 17. Örnek: Odalardan Dışarı Çıkan Robot

1. Eğitmensiz Öğrenmeye Kısa Dönüş

Hoca derse, geçen hafta direktörlükteki bir toplantı nedeniyle yapılamayan dersin telafi edileceğini söyleyerek başladı. Ardından eğitmensiz öğrenmeyi (unsupervised learning) kısaca hatırlattı.

Eğitmensiz öğrenmede elimizde örnekler vardır, ama bunların hangi sınıfa ait olduğunu bilmeyiz. Sisteme yalnız giriş bilgileri verilir, çıkışlar (etiketler) hakkında bilgi yoktur. Bu durumda yapılan şey, örnekleri benzerliklerine göre gruplamaktır. Örneğin iki özelliği olan örnekler birbirine benzerliklerine göre gruplanabiliyorsa bu bir eğitmensiz öğrenmedir. Grubun adını söylemek gerekmez, ama grupta kimlerin olduğunu biliriz.

Bu bilgiden iki türlü faydalanılır:

- **Yalnız gruplamak yeterlidir.** Örneğin müşteriler harcama bilgilerine göre gruplanır ve sonra bu gruplarla ilgili işlemler yapılır. Gruba isim vermek gerekmez; benzer kişilerin aynı grupta olduğunu görmek yeter.
- **Önce gruplayıp sonra etiketlemek.** Büyük veriyle çalışıldığında etiketleme çok zaman alır. Bazen zamandan öte bir sorun da vardır: bazı veriler herkes tarafından aynı şekilde etiketlenemez. Özellik sayısı arttıkça veriyi değerlendirmek zorlaşır. Bu yüzden önce etiketlemeden benzerliğe göre gruplanır, sonra grubun özelliklerine bakılarak grubun etiketine karar verilir.

> **Hocanın vurgusu:** Başlangıçta eğitmensiz öğrenmeye o kadar ihtiyaç duyulmuyordu, ama son yıllarda önemi iyice arttı. Bugün en çok kullanıldığı durumlardan biri "önce grupla, sonra grubun etiketine karar ver" yaklaşımıdır.

Hoca, geçen haftalarda yapay sinir ağlarında tek katmanlı ve çok katmanlı perceptron yapılarıyla insan beyninin çalışma şeklinin bilgisayarda nasıl taklit edildiğinin görüldüğünü hatırlattı. Bu derste eğitmensiz öğrenmede en çok bilinen ve en çok tercih edilen yöntemlerden biri olan öz düzenleyici haritalar (Self-Organizing Map, SOM) anlatılacaktı.

2. Rekabetçi Öğrenme ve "Kazanan Her Şeyi Alır"

SOM aslında bir rekabetçi öğrenme (competitive learning) yaklaşımıdır. Burada etiketi olmayan çıkış nöronları vardır. Çıkış nöronları giriş örneklerine benzetilmeye çalışılarak gruplama yapılır. Mantık "kazanan her şeyi alır" (winner takes all) şeklindedir.

Rekabetçi öğrenmede iki tür yaklaşım vardır:

- **Yalnız kazanan güncellenir.** Giriş örneğine en çok benzeyen çıkış nöronu kazanır ve giriş örneğine daha da benzer hâle getirilir.
- **Kazanan ve komşuları güncellenir.** Yalnız en çok benzeyen nöron değil, ona yakın olan nöronlar da girişe benzeyecek şekilde değiştirilir.

İkinci yaklaşım insan beynindeki çalışmaya benzer. Beyinde bir nöron, bağlantılı ve iletişimde olduğu başka nöronlarla birlikte çalışır. Tek bir nöronun aktif olması yerine, ona komşu nöronların da etkilenmesi sağlanır.

3. Öz Düzenleyici Haritanın (SOM) Yapısı

Hoca yapıyı tablette çizerek anlattı.

Tek boyutlu çıkış kafesi. Çıkışta 3 nöron olsun. Bu, 3 küme (cluster) istendiği anlamına gelir. Girişte 2 nöron olsun. Bu da örneklerin iki özelliği olduğu anlamına gelir. Her girişin her özelliğinden çıkıştaki her nörona bir bağlantı vardır. Bu yapıya tek boyutlu çıkış kafesi (output lattice) denir.

İki boyutlu çıkış kafesi. Komşulukları nasıl değerlendireceğinize ağı tasarlarken siz karar verirsiniz. Örneğin 9 küme isteniyorsa çıkış nöronları 3×3'lük iki boyutlu bir kafes olarak dizilebilir. Yine iki özellik olduğunu varsayalım. Yine her girişten her çıkışa bağlantı vardır.

Bu yapıda öğrenme, girişlerden çıkışlara olan ağırlıkların değiştirilmesiyle olur. Ağırlıklar güncellenerek çıkışlar girişlere benzetilir ve gruplama bu şekilde sağlanır.

4. SOM'da Öğrenme: Ağırlıkların Girişe Benzetilmesi

Hoca öğrenmenin mantığını bir giriş vektörü x ve bir ağırlık vektörü v_i üzerinden anlattı. t anında ağırlık vektörü bir yerde, giriş başka bir yerdedir. Öğrenme, $t+1$ anında ağırlığı girişe doğru yaklaştırmaktır:

$$v_i(t+1) = v_i(t) + \eta\,(x - v_i(t))$$

Burada η öğrenme oranıdır (learning rate) ve ağırlıkların ne kadar hızlı değişeceğini belirler. $x - v_i(t)$ ise giriş ile ağırlık arasındaki farktır.

Hoca bunun perceptron'daki mantıktan farklı olmadığını vurguladı: öğrenme her zaman ağırlıkların güncellenmesiyle sağlanır. SOM'daki fark, ağırlıkların giriş bilgisine benzeyecek şekilde değiştirilmesidir. Sisteme gösterilen her yeni örnek için çıkışlar girişlere benzetilecek şekilde ağırlıklar değiştirilir. Bir yakınsama (convergence) noktasında öğrenme tamamlanmış olur.

> Not: Hoca ağırlıkları bu derste de v ile gösterdi. Kısa notta aynı ağırlıklar w ile yazılmıştır.

5. SOM Öğrenme Adımları

Tabletin şarjı bittiği için hoca adımları bir süre sözlü anlattı ve öğrencilerden not almalarını istedi.

Adım 1: Giriş örneklerini normalize et. Özellikler normalize edilmezse büyük değerli özellik sonucu bastırır. Hocanın örneği: bir özellik boyu gösteriyor ve değeri 183; bir başka özellik yaşı gösteriyor ve değeri 22. Bu durumda özellikler normalize değilse büyüklüklerinden ötürü sonuçlar istenmeyen şekilde yanlış değerlendirilir. Bu yüzden girişler önce min-max ile ya da 0-1 aralığına normalize edilir.

Adım 2: Rastgele bir örnek seç. Etiketsiz giriş örneklerinden biri rastgele seçilir.

Adım 3: Benzerliği ölç. Benzerlik, girişin her çıkış nöronunun ağırlık vektörüne olan Öklid mesafesiyle (Euclidean distance) ölçülür. Girişin D özellikten oluştuğunu ve N çıkış nöronu olduğunu varsayalım:

$$x = (x_1, x_2, \dots, x_D), \qquad v_j = (v_{1j}, v_{2j}, \dots, v_{Dj}), \quad j = 1, \dots, N$$

$$d_j = \|x - v_j\| = \sqrt{\sum_{k=1}^{D} (x_k - v_{kj})^2}$$

Hoca bunu şöyle anlattı: j. nörona gelen bağlantılar $v_{1j}, v_{2j}, \dots$ ağırlıklarıdır. Örneğin birinci nörona olan mesafe, ikinci nörona olan mesafe, üçüncü nörona olan mesafe ayrı ayrı hesaplanır.

Adım 4: Kazananı bul. En küçük mesafeli nöron kazanır:

$$j^* = \arg\min_j d_j$$

Adım 5: Kazananın ağırlıklarını güncelle. Kim girişe daha çok benziyorsa onun ağırlıkları girişe daha da benzeyecek şekilde değiştirilir:

$$v_{j^*}(t+1) = v_{j^*}(t) + \eta \left(x - v_{j^*}(t)\right)$$

Adım 6: Yakınsayana kadar tekrar et. Ağırlık değerleri artık değişmiyorsa öğrenme tamamlanmıştır. O zamana kadar yeni örneklerle devam edilir.

6. Sert ve Yumuşak Öğrenme, Komşuluk Fonksiyonu

Hoca öğrenmenin iki türlü yapılabileceğini söyledi:

- **Sert öğrenme (hard learning):** Yalnız en çok benzeyen nöronun ağırlıkları değiştirilir.
- **Yumuşak öğrenme (soft learning):** Yalnız kazanan nöronun değil, onun komşularının da ağırlıkları değiştirilir.

Bu durumda "komşuluk nasıl belirlenecek?" sorusu ortaya çıkar. Hoca, yatay eksende mesafenin, dikey eksende komşuluk değerinin değişimini gösteren bir grafik çizdi.

- Sert öğrenmede komşuluk yalnız 1 ya da 0'dır: nöron kazanan j. nöronsa 1, değilse 0.
- Yumuşak öğrenmede komşuluk değeri yavaş yavaş azalır. Komşuların değerleri de birlikte değişir.

Hocaya göre komşuluk için en iyisi Gauss fonksiyonunu kullanmaktır. Kazanan nöron j^* ile j. nöron arasındaki mesafe d_{j^*j} olmak üzere:

$$h_{j^*j} = \exp\left(-\frac{d_{j^*j}^2}{2\sigma^2}\right)$$

Burada σ Gauss eğrisinin genişliğidir. Komşuluk fonksiyonu kazanan nöronda (mesafe sıfırken) en büyük değerini alır ve uzaklaştıkça azalır.

Komşuluk da işin içine katılınca ağırlık güncellemesi şöyle olur:

$$v_j(t+1) = v_j(t) + \eta\, h_{j^*j}\,(x - v_j(t))$$

Böylece kazananın yakınındaki komşuların değişmesi de güncellemeye katılmış olur. Sert öğrenme, bu formülde h'nin yalnız kazananda 1, diğer nöronlarda 0 olduğu özel durumdur.

7. Öğrenme Oranı ve Komşuluğun Zamanla Daraltılması

Komşuluğun daraltılması. Başlangıçta ağırlık değerleri genellikle rastgele belirlendiği için gerçek girişlere uzaktır. Bu yüzden güncellemelerde komşuluk geniş alınabilir, yani σ büyük seçilir. Zamanla σ küçültülür ve yalnız daha yakın komşuların değerinin değişmesi sağlanır. Adımlar ilerledikçe en çok benzeyen nöronla birlikte güncellenen komşuların sayısı azalır.

Hoca σ'nın zamanla azaltılmasını şöyle verdi: başlangıçta bir σ_0 değerinden başlanır; T_1 bir sabit, t kaçıncı iterasyonda olunduğu olmak üzere:

$$\sigma(t) = \sigma_0 \exp\left(-\frac{t}{T_1}\right)$$

Gauss eğrisi ne kadar genişse o kadar geniş bir komşuluğa bakılır. t'ye bağlı olarak zamanla daraltılınca yalnız yakın komşuluğa bakılır.

Öğrenme oranı. Hoca, öğrenme oranının backpropagation'da kullanılan öğrenme oranından farklı olmadığını söyledi. η bire ne kadar yakınsa ağırlık o kadar fazla değişir ve öğrenme o kadar hızlı olur. Ama istenen şey, ağırlığın girişe yavaş yavaş yaklaşmasıdır ki girişi geçip gitmesin. Başlangıçta ağırlık girişten çok uzak olduğu için öğrenme oranı yüksek tutulursa ikisi hızla yaklaşır. Sonra öğrenme oranı küçültülür ve ağırlık yavaş yavaş doğru yere ulaşır. Komşulukta kullanılan aynı mantıkla öğrenme oranı da zamana bağlı olarak azaltılabilir: başta büyük başlatılır, sonra küçültülür.

> **Hocanın vurgusu:** Başlangıçta ağırlıklar rastgele ve girişlerden uzaktır. Bu yüzden komşuluk geniş, öğrenme oranı büyük başlatılır; zamanla ikisi de küçültülür.

8. Örnek: SOM ile İki Kümeye Ayırma

Hoca önce 4 küme ve 2 özellik diye başladı, sonra kolay olsun diye tersini yaptı: örnekler 4 özellikli, çıkışta 2 küme var.

Verilenler:

- 4 eğitim örneği, her biri 4 özellikli.
- 2 çıkış nöronu (Unit 1 ve Unit 2), yani 2 küme.
- Kolaylık için komşuluk sıfır alınıyor: komşuluğa hiç bakılmayacak, sert öğrenme yapılacak. Yalnız kazanan çıkışın ağırlıkları değişecek.
- Öğrenme oranı ilk 4 adım için $\eta = 0{,}6$; sonra yavaş yavaş azaltılıyor.

Örnek	Özellikler
i_1	(1, 1, 0, 0)
i_2	(0, 0, 0, 1)
i_3	(1, 0, 0, 0)
i_4	(0, 0, 1, 1)

Başlangıç ağırlıkları (rastgele):

- Unit 1'e gelen ağırlıklar: $v_1 = (0{,}2;\ 0{,}6;\ 0{,}5;\ 0{,}9)$
- Unit 2'ye gelen ağırlıklar: $v_2 = (0{,}8;\ 0{,}4;\ 0{,}7;\ 0{,}3)$

Adım 1: $i_1 = (1, 1, 0, 0)$ **gösteriliyor.**

Unit 1'e mesafe:

$$(0{,}2-1)^2 + (0{,}6-1)^2 + (0{,}5-0)^2 + (0{,}9-0)^2 = 0{,}64 + 0{,}16 + 0{,}25 + 0{,}81 = 1{,}86$$

Unit 2'ye mesafe:

$$(0{,}8-1)^2 + (0{,}4-1)^2 + (0{,}7-0)^2 + (0{,}3-0)^2 = 0{,}04 + 0{,}36 + 0{,}49 + 0{,}09 = 0{,}98$$

Daha küçük mesafe Unit 2'de olduğu için kazanan (winner) Unit 2'dir. Hoca, değerlere bakıldığında da i_1'in ikinci nörona biraz daha benzediğinin görüldüğünü söyledi.

> **Ders sonrası not:** Hoca mesafe formülünü karekök içinde yazdı, ama verdiği 1,86 ve 0,98 değerleri karekök alınmamış kareler toplamıdır. Karekök alınırsa mesafeler yaklaşık 1,364 ve 0,990 olur. Karekök sıralamayı değiştirmediği için kazanan yine Unit 2'dir.

Adım 2: Unit 2'nin ağırlıkları güncelleniyor.

$$v_2(t+1) = v_2(t) + \underbrace{0{,}6\,(i_1 - v_2(t))}_{\Delta v}$$

- 1. ağırlık: $0{,}8 + 0{,}6 \cdot (1 - 0{,}8) = 0{,}8 + 0{,}12 = 0{,}92$
- 2. ağırlık: $0{,}4 + 0{,}6 \cdot (1 - 0{,}4) = 0{,}4 + 0{,}36 = 0{,}76$
- 3. ağırlık: $0{,}7 + 0{,}6 \cdot (0 - 0{,}7) = 0{,}7 - 0{,}42 = 0{,}28$
- 4. ağırlık: $0{,}3 + 0{,}6 \cdot (0 - 0{,}3) = 0{,}3 - 0{,}18 = 0{,}12$

(ara adım eklendi: her bileşenin çarpımı ayrı yazıldı.)

Yeni ağırlıklar: $v_2 = (0{,}92;\ 0{,}76;\ 0{,}28;\ 0{,}12)$.

Hoca burada şunu görmemizi istedi: i_1 girişi (1, 1, 0, 0) idi. Başlangıçta 0,8 ve 0,4 olan ilk iki ağırlık büyüyüp 0,92 ve 0,76 oldu, yani 1'lere yaklaştı. 0,7 ve 0,3 olan son iki ağırlık küçülüp 0,28 ve 0,12 oldu, yani 0'lara yaklaştı. İkinci kümenin ağırlıkları birinci örneğe benzemeye başladı. Her gelen örnek için hangi ağırlıklar bu örneğe daha çok benziyorsa, onlar örneğe daha da çok benzetilmek üzere değiştirilir.

Hoca güncellemede j'nin kazanan nöron (winning neuron) olduğunu tekrar hatırlattı. Komşuluk sıfır alınırsa yalnız kazanan nöronun ağırlıkları güncellenir; sıfır alınmazsa komşuluk fonksiyonuna göre komşular da güncellenir.

Uzun iterasyonlardan sonra ağırlıklar:

- Unit 1: $(0;\ 0;\ 0{,}5;\ 1{,}0)$
- Unit 2: $(1{,}0;\ 0{,}5;\ 0;\ 0)$

Şimdi örneklere bakıldığında i_1 ve i_3 Unit 2'ye, i_2 ve i_4 Unit 1'e daha yakındır. Kümeleme bu şekilde yapılmış olur.

Örnek	Unit 1'e mesafe (kareler toplamı)	Unit 2'ye mesafe (kareler toplamı)	Küme
i_1 (1, 1, 0, 0)	3,25	0,25	Unit 2
i_2 (0, 0, 0, 1)	0,25	2,25	Unit 1
i_3 (1, 0, 0, 0)	2,25	0,25	Unit 2
i_4 (0, 0, 1, 1)	0,25	3,25	Unit 1

(ara adım eklendi: hoca yalnız sonucu söyledi; tablo mesafeler hesaplanarak dolduruldu.)

Adım 3: Bilinmeyen yeni örnek $i_5 = (1, 1, 1, 0)$**.**

Unit 1'e mesafe:

$$(0-1)^2 + (0-1)^2 + (0{,}5-1)^2 + (1-0)^2 = 1 + 1 + 0{,}25 + 1 = 3{,}25$$

Unit 2'ye mesafe:

$$(1-1)^2 + (0{,}5-1)^2 + (0-1)^2 + (0-0)^2 = 0 + 0{,}25 + 1 + 0 = 1{,}25$$

i_5 Unit 2'ye daha çok benzer. Hoca, çok yakın olmasa da i_5'in Unit 2 kümesinde yer alabileceğini söyledi.

9. Öğrenmeden Sonra Gelen Örnek ve Kararlılık-Esneklik İkilemi

Soru: Bilinmeyen bir örnek geldiğinde Unit 2'ye ait olduğunu bulduk. Peki sistemi, yani ağırlıkları güncelliyor muyuz?

Cevap: Hoca bunun güzel bir soru olduğunu ve amaca bağlı olduğunu söyledi. Öğrenme devam ediyorsa güncellenebilir. Öğrenme tamamlanmışsa ve yalnız yeni bir örneğin kime benzediğini bulmak istiyorsanız hiçbir şeyi değiştirmezsiniz.

Hoca, bazı çalışmalarda "öğrenmeye devam etsin, yeni gelenleri de öğrensin" denildiğini, ama bunu boşu boşuna bırakmamak gerektiğini söyledi. Burada kararlılık-esneklik ikilemi (stability-plasticity dilemma) vardır:

- **Kararlılık (stability):** Öğrendiğinden çok kopmamak.
- **Esneklik (plasticity):** Yeni gelenleri de öğrenerek daha iyi genelleyebilir hâle gelmek.

Yeni gelen örneği öğrenmenin uygun olacağını düşünüyorsanız öğrenmeyi bu örneği de ekleyecek şekilde güncellersiniz. Yalnız yeni gelenin benzerliğini bulmak yeterliyse ve öğrenilmiş yapının iyi oturduğunu düşünüyorsanız öğrenmezsiniz.

10. Gauss Karışım Modeline Kısa Bakış

Hoca eğitmensiz öğrenmede Gauss karışım modelini (Gaussian Mixture Model, GMM) de anlatmayı planladığını, ama bunu dönem sonuna, vakit kalırsa bıraktığını söyledi. Bunun nedeni, GMM'nin son yıllarda daha az kullanılmasıydı: kolaylığından dolayı genelde herkes k-means kullanıyor, derin öğrenmede ise otokodlayıcılar (autoencoder) çıktı. Bu yüzden yalnız konuyu tanımanız için genel bir bilgi verdi.

Dışarıdan gördüğümüz örnekler etiketsizdir ve sınıfları belli değildir. Ama bu örnekler aslında örneğin iki Gauss dağılımından gelen, benzerliklerine göre iki kümeye dağılmış örneklerdir. Etiket olmadığı için dışarıdan karışık bir dağılım görürüz. GMM, kümelerdeki örnekleri bulup kümeleri çıkarmaya çalışır.

- GMM, k-means'e benzer bir yaklaşımla, beklenti-en büyükleme (Expectation-Maximization, EM) adımlarıyla çalışır.
- k-means'te kümeler çok belirgin biçimde ayrılır. GMM'de ise örneğin hangi tarafa ne kadar yakın olduğuna bakılır; yakınlık bilgisi de işin içine girer.
- Hocaya göre GMM, k-means'ten çok daha başarılı bir kümeleme yapar. Pek çok çalışmada önce k-means ile kümeler bulunur, sonra üzerinden GMM geçirilerek kümeleme sonuçları iyileştirilir.
- Eskiden özellikle görüntü işlemede çok kullanılırdı. Son yıllarda belki işlem yükünün fazlalığından, belki o kadar hassasiyete gerek kalmadığından daha az kullanılıyor.

  Ek bilgi (derste anlatılmadı): GMM'de her küme, kendi ortalaması ve kovaryansı olan bir Gauss dağılımıyla temsil edilir. EM algoritması iki adımı sırayla tekrarlar: E adımında her örneğin her kümeye ait olma olasılığı hesaplanır, M adımında bu olasılıklarla kümelerin ortalama, kovaryans ve ağırlıkları yeniden tahmin edilir. Bu yüzden GMM "yumuşak" bir kümeleme verir; k-means ise her örneği kesin olarak tek bir kümeye atar.

11. SOM Adımlarının Özeti

Hoca SOM'u (rekabetçi öğrenmeyi) beş adımda özetledi:

1. **İlklendirme (initialization):** Ağırlık değerlerine ilk değerler atanır.
2. **Örnekleme (sampling):** Bir x vektörü alınır, yani sisteme bir örnek gösterilir.
3. **Rekabet (competition):** x örneğinin en çok benzediği ağırlık vektörü belirlenir.
4. **Güncelleme (updating):** Kazanan nöronun ve komşularının ağırlıkları değiştirilir. Komşuluk değeri sıfır alınabilir ya da bir komşuluk fonksiyonu kullanılabilir.
5. **Uyum (adaptation):** Kazanan ve komşuları girişe daha çok benzer hâle gelir.

Bu işlem yakınsama sağlanana kadar, yani ağırlık değerleri artık belli bir miktardan fazla değişmeyene kadar, ikinci adımdan yeni bir örnek gösterilerek devam eder.

12. Pekiştirmeli Öğrenme

Aradan sonra hoca pekiştirmeli öğrenmeye (reinforcement learning) geçti. Önce diğer iki öğrenme türünü hatırlattı:

- **Eğitmenli öğrenme (supervised learning):** x girişleri ve bu girişler için çıkışların ne olacağı verilir.
- **Eğitmensiz öğrenme (unsupervised learning):** Yalnız giriş bilgileri verilir; örnekler bu bilgilere göre gruplanır.

Her ikisinde de öğrenme ağırlıklı olarak istatistiksel bilgilere, yani geçmişteki durumlara ve örneklere göre yapılır. Pekiştirmeli öğrenmede ise gerçek dünyadaki öğrenmeye daha çok benzeyen bir yapı vardır.

Köpek örneği. Bir köpeği eğittiğinizi düşünün. Sopa atıp geri getirmesini öğretmeye çalışıyorsunuz. Sopayı geri getirirse ona kemik veriyorsunuz. Köpek ödül aldığı için sopa atıldığında gidip geri getirmesi gerektiğini öğrenir: "gidip geri getirirsem ödül alacağım."

Pekiştirmeli öğrenme tamamen budur: ödül-ceza mekanizmasıyla öğrenme. Türkçede önce "ödül-ceza ile öğrenme" diye başlamıştı; bugün en çok kullanılan karşılığı "pekiştirmeli öğrenme"dir. Eğitmenli öğrenmedeki gibi tam bir etiket yoktur. Yalnız yapılan hareketlere ve işlere karşılık bir ödül ya da ceza kazanılır.

13. Pekiştirmeli Öğrenmenin Uygulamaları

Hoca pekiştirmeli öğrenmenin son yıllarda çok popüler olmasının bir nedeninin oyunlarda çok kullanılabilir olması olduğunu söyledi. Önce robotik çalışmalarda kullanılarak başlandı.

Robotik. Bir robotun bir bölgede bazı işler yapmasını istiyorsunuz, ama ona ne yapacağını tek tek söyleyemiyorsunuz; bir etiketleme yoktur. Bunun yerine "o bölgedeki şu taşı getirirsen ödül, duvara çarparsan ceza" diyerek ödül-ceza mekanizmasıyla robotu yapacağı işe yönlendirirsiniz.

E-posta ve promosyon mesajları. Bir şirket müşterilerine sürekli e-posta ya da promosyon mesajı gönderiyor. İnsanların ilgisinin azaldığını görürse bunu çok yaptığını fark eder. Hoca, pek çoğumuzun çok fazla mesaj bombardımanına maruz kalınca abonelikten çıktığını (unsubscribe) söyledi. Firma müşteri kaybetmeye başladığını görünce mesaj bombardımanını keser. Bu, gerçekten pekiştirmeli öğrenmeyle yapılan bir şeydir.

Dijital reklam açık artırması. Hoca bunun yanılmıyorsa Adobe'nin ve Alibaba'nın yaptığı uygulamalardan biri olduğunu, ama tam hatırlamadığını söyledi.

- Sayfalarda gezinirken örneğin "spor ayakkabı" diye arama yaparsınız ve sağ köşede bir Nike reklamı görmeye başlarsınız.
- Reklamın gösterilmesi şöyle olur: firmalar önce hedef kullanıcıyı belirler. Aradığınız kelimelerden ve genel aramalarınızdan profiliniz çıkarılır. Örneğin basketbolla ilgilendiği belli olan birine basketbol ayakkabısı, bir başkasına sporla çok ilgisi olmayan daha moda spor ayakkabılar gösterilir.
- Reklam verenler sayfalardaki reklam yerleri için açık artırmaya girer. Kim daha iyi fiyat veriyorsa onun reklamı görünür. Örneğin Nike "şu tür kullanıcılara reklamımı göstermek istiyorum" der ve açık artırmaya 100 lirayla girer.

- Bir süre sonra ürünü alan olmuyorsa stratejisini değiştirmesi gerekir. Belki belirlediği hedef kitle doğru değildir, belki pazarlama zamanı doğru değildir. Hocanın örneği: şu anda spor salonları kapalı, insanlar oynamıyor; o reklamı vermenin anlamı yok.
- Burada bir ceza vardır: ürün alınmıyor. O zaman ya açık artırmadan çekilmek ya da ödenen parayı düşürmek gerekir (örneğin 100 lira yerine 50 lira). Tersine, çok iyi satış alınıyorsa bu fırsatı kaçırmamak; satış alınabilecekken açık artırmada önerilen para yetmiyorsa onu artırmak gerekir.

Hoca bu örneği eğitmenli öğrenmeyle yapmanın pek mümkün olmadığını vurguladı: burada zamana göre değişen durumda "olmuyorsa şöyle yap, oluyorsa böyle yap" diyen bir ödül-ceza mantığı vardır.

> **Hocanın vurgusu:** Pekiştirmeli öğrenme, şartlara göre ödül ve cezayla ne yapacağına karar vermesi bakımından gerçek insan öğrenmesine en çok benzeyen yaklaşımdır. Yayınlarda da yıldızının gittikçe parladığı görülüyor.

14. Durum, Eylem ve Ödül

Pekiştirmeli öğrenmede üç bileşen vardır. Gerçek dünyada (ortamda) bir robotumuz olduğunu varsayalım:

- **Durum (state, gözlem/observation):** Robot bir durumdadır.
- **Eylem (action):** Bulunduğu duruma göre bir eylem yapar.
- **Ödül (reward):** Yaptığı işin karşılığında ödül ya da ceza gelir.

Robot önce hangi durumda olduğuna bakar, sonra bu durumda ne yapmasının doğru olacağına karar verir. Örneğin duvara çarptıysa geri döner; yerdeki taşa ulaştıysa onu alır. Eylemi yapınca yeni bir duruma geçer ve bu yeni durumun uygunluğuna göre ödül ya da ceza alır.

Pekiştirmeli öğrenme yinelemeli (iteratif) bir süreçtir. Ödül ya da cezaya göre öğrenme gerçekleşir ve öğrenme tamamlanana kadar devam edilir. Öğrenmenin tamamlanması, belli bir noktadan sonra ne yapılacağının öğrenilmiş olmasıdır; köpeğin atılan sopayı gidip geri getirmesi gerektiğini öğrenmesi gibi.

15. Markov Karar Süreci

Pekiştirmeli öğrenmede kullanılan yaklaşım Markov karar sürecidir (Markov Decision Process, MDP).

Bir kararı verirken o kararı etkileyen, geçmişte belli bir süre yapılmış işler vardır. Ama bunların hepsi değerlendirilmeye kalkılırsa çok büyük bir olasılık hesabı yapmak gerekir: t anını $t-1$ etkiler, $t-1$'i $t-2$ etkiler ve geçmişe gittikçe hesap çok büyür. Bu yüzden şu varsayım yapılır: 1'den t'ye kadar olan bütün geçmiş, yalnız t anındaki durum s_t ile gösterilebilir ve $t+1$ anındaki durumu yalnız t anı etkiler. Yani daha eski durumlar dikkate alınmaz, yalnız bir önceki durum dikkate alınır.

MDP'nin dört parametresi vardır:

- S: ortamın durumları (states).
- A: yapılabilecek eylemler (actions).
- T: durum geçiş fonksiyonu (state transition function). Bir durumdan diğerine nasıl geçileceğini belirler. s_{t+1} yalnız t anındaki duruma bağlıdır.
- R: anlık ödül fonksiyonu (immediate reward). t anında bir eylem yapılır ve $t+1$ anında hemen onun ödülü ya da cezası alınır.

  Ek bilgi (derste anlatılmadı): Markov varsayımı genellikle $P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, ...) = P(s_{t+1} \mid s_t, a_t)$ biçiminde yazılır. Yani bir sonraki durum, geçmişin tamamına değil yalnız şimdiki duruma ve yapılan eyleme bağlıdır. Hocanın sözlü anlattığı "sadece T anı T artı bir anını etkiler" ifadesinin matematiksel karşılığı budur.

16. Q-Öğrenme

Hoca pekiştirmeli öğrenmede bugün en çok kullanılan öğrenme modellerinden biri olan Q-öğrenmeyi (Q-learning) anlattı. Q-öğrenme derin öğrenmede de kullanılan bir yaklaşımdır ve

kolay olduğu için en çok kullanılan pekiştirmeli öğrenme yöntemidir.

Mantık şudur: bir durumdayken yapılabilecek eylemler arasından en yüksek getirisi olanı seçmek. Her zaman yapılmaya çalışılan budur. Q'nun "Q"su kaliteden (quality) gelir; her durum için bir sonraki eylemin en iyi sonucu getiren eylem olması hedeflenir.

İki matris vardır:

- **Ödül matrisi R:** Anlık ödülü gösterir. Bir durumdayken yapılan eylemin ödülü ya da cezası nedir, onu verir. Tasarımcı belirler.
- **Q matrisi:** Bir noktadan bir noktaya gitmenin gerçekte, deneyimle öğrenilmiş ödül-ceza değerini tutar. Öğrenilmeye çalışılan budur.

Hocanın derste kullandığı güncelleme kuralı:

$$Q(s,a) = R(s,a) + \gamma \cdot \max_{a'} Q(s',a')$$

Burada s bulunulan durum, a yapılan eylem, s' eylemin götürdüğü yeni durum, a' yeni durumda yapılabilecek eylemlerdir. Yani bulunulan durumda yapılan eylemin değeri, anlık ödül ile bir sonraki durumda yapılabilecek en iyi eylemin değerinin bir katsayıyla çarpımının toplamıdır. Hoca bu katsayıya "öğrenme oranı" ve "learning parametresi" dedi ve örnekte $0{,}8$ aldı.

> **Ders sonrası not:** Bu formüldeki $0{,}8$'lik katsayı, standart terminolojide öğrenme oranı değil indirim katsayısıdır (discount factor, γ). Gelecekteki ödüllerin bugüne ne kadar yansıyacağını belirler. Öğrenme oranı (α) içeren genel Q-öğrenme kuralı $Q(s,a) \leftarrow Q(s,a) + \alpha\left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$ biçimindedir. Derste kullanılan basit hâl, bu kuralda $\alpha = 1$ alınmasına karşılık gelir.
>
> **Hocanın vurgusu:** Ödülün ve cezanın ne olacağına ve büyüklüğüne siz karar verirsiniz. Öğrenme parametresi ve ödül matrisi, kullanan kişi tarafından deneyimle belirlenmesi gereken şeylerdir.

17. Örnek: Odalardan Dışarı Çıkan Robot

Problem. Bir robot, bulunduğu noktadan dışarı çıkmaya çalışıyor. Kapalı odalar var: A, B, C, D, E. F durumu dışarısı, yani hedeftir. Durum, bulunulan odadır; eylem, bir odadan diğerine geçmektir.

Hocanın anlattığı kapı bağlantıları:

- A ile E arasında kapı var. A ile B arasında kapı yok.
- B'den doğrudan F'ye (dışarı) çıkılıyor. B ile D arasında kapı var.
- C kapalı bir oda; yalnız D'ye açılıyor.
- D, B, C ve E ile bağlantılı.
- E'den doğrudan F'ye çıkılıyor; E ayrıca A ve D ile bağlantılı.
- F'den F'ye (dışarıda kalmak) de bir geçiş olarak tanımlanıyor.

Ödül matrisi R. Ödül, hedefe ulaşmaktır. Doğrudan çıkışa (F'ye) giden kapıların ödülü 100, diğer kapıların ödülü 0 verilir. Bağlantı olmayan geçişler "null" (tanımsız) olarak işaretlenir. Hoca bu matrisin aslında odaların oluşturduğu grafın bağlantı matrisi olduğunu söyledi: düğümler odalar, kenarlar kapılardır.

R	A	B	C	D	E	F
A	–	–	–	–	0	–
B	–	–	–	0	–	100
C	–	–	–	0	–	–
D	–	0	0	–	0	–
E	0	–	–	0	–	100
F	–	0	–	–	0	100

("–": bağlantı yok, null.)

Başlangıç. Q matrisi sıfırla ilklendirilir. Katsayı $0{,}8$ alınır.

1. bölüm (episode). Başlangıç durumu rastgele B seçilsin. B'deyken rastgele F'ye gitmek seçilsin.

$$Q(B,F) = R(B,F) + 0{,}8 \cdot \max\{Q(F,B),\ Q(F,E),\ Q(F,F)\} = 100 + 0{,}8 \cdot 0 = 100$$

F'den gidilebilecek yerler B, E ve F'dir; Q matrisi henüz başlangıç hâlinde olduğu için hepsi 0'dır. Böylece $Q(B,F) = 100$ olur. F'ye ulaşıldığı için bu bölüm biter.

> **Ders sonrası not:** Hoca bu hesapta önce sonucu yanlışlıkla 80 yazdı, sonra kendisi düzeltti: $0{,}8$'i 1 ile çarptığını sandığını söyledi. Doğru değer 100'dür. Burada cezalar 0 alındığı için hiçbir değer negatif çıkmaz.

2. bölüm. Yeni bir rastgele başlangıç durumu D olsun. D'den gidilebilecek üç yer var: B, C ve E. Rastgele B seçilsin.

$$Q(D,B) = R(D,B) + 0{,}8 \cdot \max\{Q(B,D),\ Q(B,F)\} = 0 + 0{,}8 \cdot \max\{0,\ 100\} = 80$$

D'den B'ye gitmenin anlık ödülü 0'dır, ama B'den F'ye gitmenin öğrenilmiş değeri 100 olduğu için D'den B'ye gitmenin değeri 80 olarak güncellenir. Hoca, Q matrisinin deneyimlerden öğrenileni güncel olarak sakladığını ve uzun vadede bulunulan bir noktadan hedefe ulaşmanın değerini öğrenmeye çalıştığını söyledi.

Bölüm henüz bitmedi; robot B'de. Hoca burada önemli bir ayrıntıyı vurguladı: değeri hesaplarken bir sonraki durumdaki eylemlerin **en büyüğü** kullanılır, ama bölüm ilerletilirken bir sonraki eylem yine olası seçenekler arasından **rastgele** seçilir. Çünkü hangi yolun sonuca ulaştıracağı bilinmediği için gelişigüzel yollar denenir. B'den F'ye gitmek seçilsin:

$$Q(B,F) = 100 + 0{,}8 \cdot \max\{Q(F,B),\ Q(F,E),\ Q(F,F)\} = 100 + 0{,}8 \cdot 0 = 100$$

$Q(B,F)$ zaten 100'dü; değişmez. F'ye ulaşıldığı için bölüm biter.

Yakınsamış Q matrisi. Uzun öğrenmeler sonunda Q matrisi şu hâle gelir:

Q	A	B	C	D	E	F
A	–	–	–	–	80	–
B	–	–	–	64	–	100
C	–	–	–	64	–	–
D	–	80	51	–	80	–
E	64	–	–	64	–	100
F	–	80	–	–	80	100

Hoca bu matrisin normalize edilmiş olduğuna dikkat çekti: uzun bölümler sonunda değerler 100'ü geçer; elle hesaplanırsa değerlerin büyüdüğü görülür. Tablodaki değerler, en büyük değer 100'e getirilecek şekilde normalize edilmiş hâlidir.

(ara adım eklendi: güncelleme kuralı yakınsayana kadar tekrarlanınca en büyük değer $Q(F,F) = 100 + 0{,}8 \cdot Q(F,F)$ eşitliğinden 500 çıkar. Bütün değerler 500'e bölünüp 100 ile çarpılınca tablodaki değerler elde edilir; örneğin $Q(D,C) = 0 + 0{,}8 \cdot Q(C,D)$ ve $Q(C,D) = 0{,}8 \cdot Q(D,B) = 0{,}8 \cdot 400 = 320$ olduğundan $Q(D,C) = 256$, normalize hâli $51{,}2 \approx 51$. Değerler python ile tekrar hesaplandı ve tablo ile tuttu.)

Hoca tabloyu oda planıyla birlikte yorumladı:

- D'den C'ye gitmenin sağladığı kazanç 51'dir, çünkü C kapalı bir odadır ve oraya gitmenin hiçbir anlamı yoktur.
- D'den B'ye gitmenin kazancı 80'dir.
- E'den A'ya gitmek 64'tür, çünkü A'da kapanıp kalınır; ama A'dan E'ye gitmek 80'dir.

Öğrenilmiş Q ile yol bulma. Robot C'de olsun. Dışarı çıkmak için her durumda en büyük değerli eylemi seçer:

1. C'deyken gidebileceği tek yer D'dir ($Q(C, D) = 64$). Birden fazla seçenek olsaydı da D'ye giderdi.
2. D'deyken B'ye (80) ya da E'ye (80) gidebilir. C'ye gitmenin (51) anlamı yoktur. İkisi eşit olduğu için biri seçilir; B seçilsin.
3. B'deyken D'ye (64) ya da F'ye (100) gidebilir. F büyük olduğu için F'ye gider.

Yol: C → D → B → F. Robot çıkışa ulaşmış olur. Yakınsamadan sonra bulunulan durumdan bir sonraki duruma geçerken en yüksek değerli eylem seçilir; böylece olası yollar arasından en uygunu, yani en kısa yol bulunur.

> **Ders sonrası not:** Hoca yolu toparlarken "C'den B'ye gitti, D'den B'ye gitti, B'den de F'ye gitti" dedi. C ile B arasında kapı yoktur; doğru yol C → D → B → F'dir. Hocanın adım adım anlatımı da bu yolu verir.

Q-öğrenme algoritmasının özeti. Tablet kapandığı için hoca son kısmı sözlü tamamladı:

```
1. Öğrenme parametresini (katsayıyı) ve ödül matrisi R'yi belirle.
2. Q matrisini sıfırla ilklendir.
3. Q matrisindeki değerler artık değişmeyene (yakınsayana) kadar her bölüm için:
   a. Rastgele bir başlangıç durumu seç.
   b. Hedefe ulaşana kadar:
      - Bulunulan durumdaki olası eylemlerden birini rastgele seç.
      - Bu eylemin Q değerini Q(s,a) = R(s,a) + 0,8 * max Q(s',a') ile hesapla
        ve Q matrisini güncelle.
      - Eylemin götürdüğü duruma geç.
4. Öğrenme bitince her durumda en büyük Q değerli eylemi seçerek hedefe git.
```

Dersin Toparlaması

Bu derste eğitmensiz öğrenmenin en bilinen yöntemlerinden biri olan öz düzenleyici haritalar anlatıldı. SOM bir rekabetçi öğrenme yöntemidir: her girişe en çok benzeyen çıkış nöronu kazanır ve onun ağırlıkları $v(t+1) = v(t)+\eta\,(x-v(t))$ kuralıyla girişe yaklaştırılır. Sert öğrenmede yalnız kazanan, yumuşak öğrenmede Gauss komşuluk fonksiyonuyla kazananın komşuları da güncellenir. Başlangıçta komşuluk geniş ve öğrenme oranı büyük tutulur; ikisi de zamanla küçültülür. Dört özellikli dört örnekle yapılan sayısal örnekte ağırlıklar yakınsayınca örnekler iki kümeye ayrıldı ve yeni bir örneğin hangi kümeye benzediği mesafeyle bulundu. Öğrenme sonrası gelen örnekle öğrenmeye devam edilip edilmeyeceği amaca bağlıdır ve kararlılık-esneklik ikilemine dikkat etmeyi gerektirir. Gauss karışım modeli ise yalnız kısaca tanıtıldı.

Dersin ikinci yarısında pekiştirmeli öğrenmeye geçildi. Etiket yerine eylemlere karşılık ödül ya da ceza vardır; ajan durum-eylem-ödül döngüsüyle ne yapacağını öğrenir. Bu süreç Markov karar süreciyle modellenir: bir sonraki durum yalnız şimdiki duruma bağlıdır. Q-öğrenmede anlık ödül matrisi R ile başlanır ve rastgele bölümlerle Q matrisi güncellenerek her durumdan hedefe ulaşmanın değeri öğrenilir. Odalardan çıkan robot örneğinde yakınsamış Q matrisiyle C odasından başlayan robot C → D → B → F yolunu buldu. Hoca, tablet ve bilgisayar şarj sorunları nedeniyle aksayan dersin Q-öğrenme kısmının üzerinden önümüzdeki hafta tekrar geçileceğini söyledi.

Ders Sonrası Notlar

> **Ders sonrası not:** Gauss komşuluk fonksiyonu sözlü anlatımda "e üzeri eksi d kare bölü 2 d kare" diye geçiyor. Paydadaki ifade Gauss eğrisinin genişliği olan σ'nın karesidir: $h = \exp(-d^2/(2\sigma^2))$. Hoca da hemen ardından genişliğin σ olduğunu ve zamanla küçültüldüğünü anlatıyor.

> **Ders sonrası not:** Hoca GMM'nin "k-means'ten çok daha başarılı" kümeleme yaptığını söyledi. Bu her veri için geçerli değildir; GMM, kümeler Gauss dağılımına uyduğunda ve kümelerin biçimi ya da büyüklüğü farklı olduğunda üstünlük sağlar. Başarı veriye bağlıdır.

Ders sonrası not: Altyazıda bozuk geçen terimlerin doğru yazımları: “self-organizing MEP” = self-organizing map, “kompetitif” = competitive, “Goals/GOS/GAUSE fonksiyonu” = Gauss fonksiyonu, “Neybrut/N-Brut” = neighborhood (komşuluk), “binarımız/binik nöron” = winner (kazanan) nöron, “lip distance” = Öklid (Euclidean) distance, “Gözlümünün mixture/Gözün Mixture/gözsün x-chim model” = Gaussian mixture model, “Kavniis” = k-means, “iştenmikinin” = işlem yükünün, “Re-Course Learning” = reinforcement learning, “dual-sporn-wise learning” = supervised learning, “Markle Decision proses” = Markov decision process, “zihir bazların” = (anlaşılmıyor; robotun çalışacağı bir bölge), “Q-Learnik” = Q-learning, “nal” = null, “Ali Baba” = Alibaba (şirket).

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.